

**World College of Technology and Management**

Department of Computer Science & Engineering

# **INDUSTRIAL INTERNSHIP REPORT**

ON

## **NovaBank – Banking Information System**

*Core Java Internship Project*

**Prepared by**

**Krashanu**

Internship Domain: Core Java

upskill Campus • The IoT Academy • UniConverge Technologies Pvt Ltd

# TABLE OF CONTENTS

---

|                                                  |    |
|--------------------------------------------------|----|
| 1 Preface .....                                  | 3  |
| 2 Introduction .....                             | 4  |
| 2.1 About UniConverge Technologies Pvt Ltd ..... | 4  |
| 2.2 About upskill Campus .....                   | 8  |
| 2.3 The IoT Academy / Objectives .....           | 9  |
| 2.4 Reference .....                              | 10 |
| 2.5 Glossary .....                               | 10 |
| 3 Problem Statement .....                        | 11 |
| 4 Existing and Proposed solution .....           | 12 |
| 4.1 Code submission (Github link) .....          | 12 |
| 4.2 Report submission (Github link) .....        | 12 |
| 5 Proposed Design/ Model .....                   | 13 |
| 5.1 High Level Diagram .....                     | 13 |
| 5.2 Low Level Diagram .....                      | 13 |
| 5.3 Interfaces .....                             | 13 |
| 6 Performance Test .....                         | 14 |
| 6.1 Test Plan/ Test Cases .....                  | 14 |
| 6.2 Test Procedure .....                         | 14 |
| 6.3 Performance Outcome .....                    | 14 |
| 7 My learnings .....                             | 15 |
| 8 Future work scope .....                        | 16 |

# 1 Preface

---

This Industrial Internship Report presents the work completed during my Core Java internship on the project “NovaBank – Banking Information System”. The internship work was carried out as a practical software development project in which I moved from understanding the requirements and planning the application structure to implementing, integrating, testing and documenting a complete console-based banking system.

The project was developed using Core Java and Object-Oriented Programming concepts. During the internship, I worked with multiple classes and packages for customer, account and transaction management, application services, local data storage and utility functions. The system was developed through weekly milestones, beginning with project structure and model classes, followed by customer registration, authentication, account operations and banking transactions. The later phase focused on integration, debugging, validation, end-to-end testing and final code review.

Relevant industrial internship experience is important for connecting academic concepts with practical software development. This project helped me understand how individual Java classes communicate in a real application and how design decisions, validation and testing affect the complete workflow. It also gave me practical experience in resolving compilation, package and method-integration issues.

The internship opportunity provided through upskill Campus and The IoT Academy in association with UniConverge Technologies Pvt Ltd gave me a structured environment to work on a defined project and document the development process. The program was approached week by week, with each stage building on the previous stage and ending with testing and review.

I would like to thank the internship program team, mentors and faculty members who provided guidance and support during the project. I would also thank my peers and everyone who helped directly or indirectly during development and testing.

My message to juniors and peers is to focus on understanding the fundamentals, build projects step by step, and not be discouraged by compilation or integration errors. Debugging and repeated testing are an important part of learning software development.

## 2 Introduction

---

### 2.1 About UniConverge Technologies Pvt Ltd

UniConverge Technologies Pvt Ltd (UCT) is described in the internship report format as an organization working in the Digital Transformation domain and providing industrial solutions with a focus on sustainability and return on investment. The sample internship material describes the use of technologies such as Internet of Things (IoT), Cyber Security, Cloud computing, Machine Learning, communication technologies including 4G/5G/LoRaWAN, Java Full Stack, Python and front-end technologies.

The sample also describes UCT's IoT platform, UCT Insight, as a platform intended for rapid deployment of IoT applications and business/process insight. It is described as using Java for the backend and ReactJS for the front end, with support for MySQL and NoSQL databases. The platform supports device connectivity through industrial protocols such as MQTT, CoAP, HTTP, Modbus TCP and OPC UA, and can be deployed in cloud or on-premises environments.

These descriptions provide the industrial context in which the internship program was organized. My assigned work was focused specifically on Core Java and the development of the NovaBank Banking Information System.

## 2 Introduction – Industrial Context

---

The internship report format provides an industrial context for UniConverge Technologies Pvt Ltd and its technology work. The sample material describes additional solutions such as smart factory monitoring, production and asset monitoring, OEE and predictive maintenance, along with LoRaWAN-based solutions for areas including Agritech, smart cities and industrial monitoring.

These examples show the broader technology environment associated with the internship program. My own project was narrower in scope and concentrated on Core Java application development rather than IoT deployment. The main learning objective was to apply Java programming and software design concepts to a complete functional system.

### Project Context

- Project: NovaBank – Banking Information System
- Domain: Core Java
- Application type: Console/menu-driven banking system
- Main development focus: OOP, collections, validation, local persistence, integration and testing

## 2 Introduction – Project Context

---

NovaBank was developed as a practical Core Java application to represent a small banking information workflow. The project was organized so that the application logic was not concentrated in a single class. Instead, customer, account and transaction information was represented through model classes, while the BankService class handled the main operations.

The project also used DataStorage for local persistence through Java serialization and IdGenerator for generating unique customer and account identifiers. The Main class connected the service layer with the menu-driven console interface. This structure made it possible to develop and test the application in stages.

### Major Components

- Customer – customer-related information and profile operations.
- Account – account information and balance-related data.
- Transaction – transaction-related information.
- Admin – model foundation for administrative functionality.
- BankService – main customer, account and transaction operations.
- DataStorage – local persistent storage using Java object serialization.
- IdGenerator – unique customer IDs and account numbers.
- Main – menu-driven application flow.

## 2 Introduction – Development Progress

---

The first stage of development focused on understanding the problem statement, planning the package structure and creating the core model and utility classes. The next stage converted this foundation into a functional banking application by implementing customer registration, unique identifiers, login, account operations and transactions.

The third stage concentrated on integration. Main.java, BankService and the model classes were connected, and compilation, package and method-signature issues were resolved. The final stage focused on completing the major workflows, performing end-to-end testing, debugging remaining issues, reviewing the code structure and preparing the project for final demonstration and documentation.

### Development Flow

- Requirement understanding and project planning
- Package and model structure creation
- Customer and account functionality
- Authentication and menu-driven workflow
- Deposit, withdrawal and fund transfer
- Transaction history and validation
- Integration and debugging
- End-to-end testing and final review

## 2.2 About upskill Campus

---

upskill Campus (USC), along with The IoT Academy and in association with UniConverge Technologies Pvt Ltd, facilitated the internship process. The sample internship material describes upskill Campus as a career development platform designed to provide personalized executive coaching and career-oriented learning support.

For this internship, the platform provided the structure for project-based learning, weekly progress reporting and final project documentation. The internship helped connect technical learning with practical implementation by requiring the project to be developed, tested and documented as a complete application.

## 2.3 The IoT Academy / Objectives

---

### The IoT Academy

The provided sample report describes The IoT Academy as the EdTech division of UCT that runs long executive certification programs in collaboration with EICT Academy and IITs across multiple domains. In the internship process, it forms part of the ecosystem through which the project-based learning and internship activities were facilitated.

### 2.4 Objectives of this Internship Program

The internship objectives, as reflected in the provided internship report format, were to gain practical industry experience, work on real-world problems, improve understanding of the relevant technical field and its applications, improve job prospects, and develop personal skills such as communication and problem solving.

For my project, these objectives were addressed through practical Core Java development. I worked with object-oriented design, packages, collections, validation, local persistence, debugging, integration and end-to-end testing. The project also required maintaining a structured repository and preparing the implementation for final demonstration and documentation.

## 2.5 Reference

---

1. Oracle Java Documentation – Core Java language and API concepts.
2. Core Java and Object-Oriented Programming learning resources used during the internship.
3. Java Collections and Exception Handling tutorials used for implementation and debugging.
4. Visual Studio Code Java Extension Pack documentation and Java development resources.
5. GitHub repository for the NovaBank project: <https://github.com/krashanu6-sudo/upskillcampus>

## 2.6 Glossary

| Terms | Acronym / Meaning                 |
|-------|-----------------------------------|
| OOP   | Object-Oriented Programming       |
| ID    | Identifier                        |
| API   | Application Programming Interface |
| UI    | User Interface                    |
| CRUD  | Create, Read, Update and Delete   |
| I/O   | Input/Output                      |
| UML   | Unified Modeling Language         |

**Project repository:** <https://github.com/krashanu6-sudo/upskillcampus>

### 3 Problem Statement

---

The problem addressed by the project was to develop a simple Banking Information System using Core Java that could manage customer information, accounts and basic banking transactions through a structured application workflow.

A basic manual or loosely organized approach to banking records can make it difficult to maintain consistent customer data, account balances and transaction information. For an academic or training environment, the challenge is to model these entities clearly and connect them through reusable application logic while also validating user input.

The NovaBank project therefore focuses on creating a modular console-based system in which customer registration, login, account operations and transactions are connected through Java classes and a service layer. Local persistence was included using Java object serialization so that relevant application data could be maintained between runs.

## 4 Existing and Proposed solution

---

A basic approach to a small banking-record application can rely on direct input handling and manually maintained information. Such an approach is useful for demonstrating individual operations, but it can become difficult to maintain when customer, account and transaction information grows or when multiple operations depend on the same data.

The proposed NovaBank solution separates responsibilities into model, service, storage and utility components. Customer, Account, Transaction and Admin are represented as model classes. BankService contains the main business operations, DataStorage provides local persistent storage through Java serialization, and IdGenerator supports unique customer and account identifiers. The Main class provides the menu-driven console workflow.

The value addition of the proposed solution is the combination of these components into one connected Core Java application. It supports customer registration and login, profile management, balance checking, deposits, withdrawals, transfers, transaction history/mini statements, password management, account search and validation of important inputs.

### 4.1 Code submission (Github link)

<https://github.com/krashanu6-sudo/upskillcampus>

### 4.2 Report submission (Github link)

[https://github.com/krashanu6-sudo/upskillcampus/blob/main/BankingInformationSystem Krashanu USC UCT Official.pdf](https://github.com/krashanu6-sudo/upskillcampus/blob/main/BankingInformationSystem%20Krashanu%20USC%20UCT%20Official.pdf)

## 5 Proposed Design/ Model

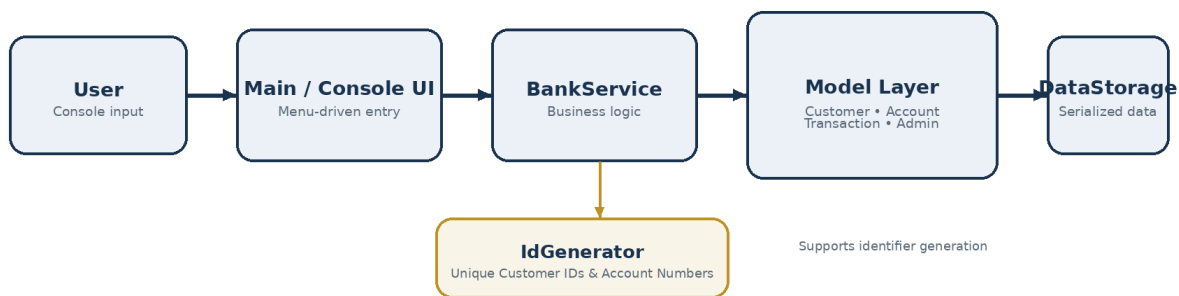
The application follows a modular flow. A user interacts with the Main class through the console. Menu choices are passed to BankService, which performs the required operation on the relevant model objects. DataStorage is used for persistent local storage, while IdGenerator supports unique identifiers.

Core modules: model (Customer, Account, Transaction, Admin); service (BankService); storage (DataStorage); util (IdGenerator); application entry (Main); submission entry (BankingInformationSystem.java).

The design uses encapsulation and separation of responsibilities. Customer and account information is represented as objects, service methods handle banking operations, and storage is kept separate from the application flow. Validation is applied before important banking changes such as withdrawals and transfers.

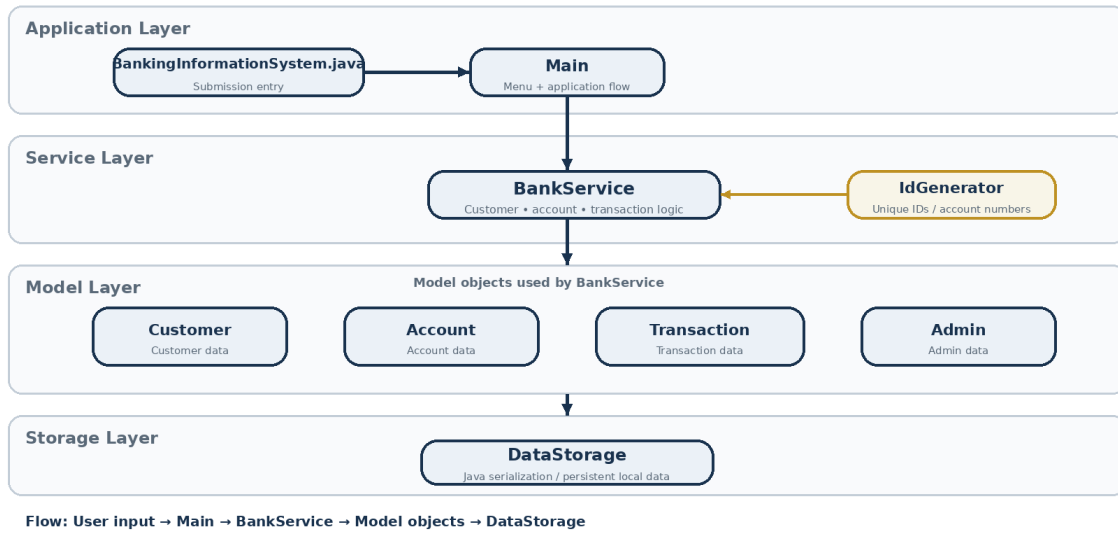
### 5.1 High Level Diagram

#### High Level System Architecture



### 5.2 Low Level Diagram

## Low Level Component / Data Flow



### 5.3 Interfaces

The primary interface is a menu-driven console. The customer dashboard provides options for profile viewing, balance checking, profile update, deposit, withdrawal, fund transfer, mini statement, password change and logout. The application also provides account search and listing functions from the main menu.

## 6 Performance Test

---

Testing was performed as functional and end-to-end testing through the menu-driven console application. The focus was on validating the main workflows, invalid inputs and the interaction between connected classes. The project was repeatedly compiled and executed after corrections.

The internship work did not include a controlled benchmark of throughput, latency, memory consumption or production-scale concurrent users. Therefore, no unsupported numerical performance claim is made here. Instead, the performance-related constraints considered were functional correctness, data consistency, successful compilation/execution and reliable handling of invalid banking operations.

### 6.1 Test Plan/ Test Cases

| No. | Test Case             | Input / Condition                     | Expected / Observed Result                                      |
|-----|-----------------------|---------------------------------------|-----------------------------------------------------------------|
| 1   | Customer registration | Valid customer details                | Customer is registered and an account is created                |
| 2   | Customer login        | Valid credentials                     | Customer dashboard is opened                                    |
| 3   | Profile update        | Updated profile information           | Profile information is updated                                  |
| 4   | Deposit               | Valid amount                          | Account balance and transaction information are updated         |
| 5   | Withdrawal            | Valid amount within available balance | Withdrawal is processed                                         |
| 6   | Withdrawal validation | Invalid/excess amount                 | Operation is rejected through validation                        |
| 7   | Fund transfer         | Valid source, destination and amount  | Transfer is processed and transaction information is maintained |
| 8   | Mini statement        | Customer with transaction history     | Relevant transaction information is displayed                   |
| 9   | Password change       | Valid new password details            | Password is updated                                             |

## 6.2 Test Procedure

The test procedure was carried out in stages. Individual functions were first checked during implementation. After integration, the complete application was executed through the console and different user workflows were repeated. Valid and invalid inputs were used for important banking operations. Compiler messages and runtime behaviour were used to identify problems. After each correction, the application was compiled and executed again to verify that connected modules continued to work correctly.

The final week included end-to-end testing from the user's perspective, final debugging, code review and verification that the project compiled successfully after the changes.

## 6.3 Performance Outcome

The final testing outcome was that the major customer, account and transaction workflows were implemented, integrated and tested as a complete console application. The final review verified customer registration, login, account operations, deposit, withdrawal, fund transfer, transaction history/mini statement and related validation.

The main constraint identified during development was integration complexity: changes in one class could affect connected classes. This was handled through modular organization, consistent method signatures, repeated compilation and feature-by-feature testing followed by complete workflow testing.

Because the project was developed as a Core Java internship application rather than a production banking platform, scalability, high-volume concurrency, network security and production-grade database performance were outside the tested scope.

## 7 My learnings

---

The internship strengthened my practical understanding of Core Java and Object-Oriented Programming. I learned how to plan a multi-class application, separate responsibilities through packages and connect model objects with a service layer.

I gained experience with encapsulation, constructors, getters and setters, collections, validation, exception handling, Java serialization and utility classes. I also improved my ability to debug compilation errors, package-related issues and method-signature mismatches. The project showed me that successful compilation is not enough; each feature must be tested after integration because a change in one class can affect other connected components.

From a career perspective, the internship improved my confidence in reading and modifying a Java codebase, understanding application flow, debugging systematically and explaining technical implementation. These are practical foundations that can be carried into larger software projects.

## 8 Future work scope

---

The current project is a console-based Core Java application with local serialized storage. The following improvements could be considered as future work:

- Replace local Java serialization with a relational database such as MySQL or PostgreSQL.
- Add a graphical or web-based user interface instead of a console-only interface.
- Introduce stronger authentication and secure password storage.
- Expand the admin module with controlled administrative operations and reporting.
- Add more detailed account and transaction reports.
- Add automated unit and integration tests to reduce manual testing effort.
- Improve exception handling and validation for a wider range of edge cases.
- Add logging and audit trails for important banking operations.
- If the system is extended for multi-user or networked use, introduce appropriate concurrency controls, security measures and scalable data access.

These items were not required to complete the internship scope and can be explored in a future version.